

startup.S × linker.ld × app.elf 三方对照

- RT-Thread Nano on ESP32-S31: [startup.S × linker.ld × app.elf 三方对照](#)
 - 0. 先建立直觉
 - 1. 三份“图纸”的分工
 - 2. 第一站：镜像怎么进到 RAM (app.bin 的头、段、搬运)
 - 3. 第二站：_start 逐条对照 (源码 ↔ 反汇编 ↔ 为什么)
 - 4. 第三站：linker.ld 逐段对照
 - 5. 第四站：符号表怎么读 (nm 的字母、addr2line 反查)
 - 6. 第五站：gp 与小数据 —— 唯一一处“编译器 × 链接器 × 启动文件”的契约
 - 7. 第六站：这套约定由谁保护
 - 8. 三次真实事故复盘 (都用 ELF/map 定位)
 - 附录 A：一页速查 (地址 ↔ 符号 ↔ 谁给的)
 - 附录 B：复现本文所有数据的命令
 - 附录 C：术语表
 - 附录 D：本次核对的环境与产物

RT-Thread Nano on ESP32-S31: startup.S × linker.ld × app.elf 三方对照

目标：把“启动文件、链接脚本、最终生成的 ELF”这三样东西**对上号**——你看得见源码里的每一行，也看得见它在内存里的哪一个地址、变成了哪几条机器码、为什么必须排在那儿。

本文**所有地址、大小、字节、反汇编**都来自本工程**真实构建产物**，不是示意值。复现命令见附录 B。

项目	值
工程	projects\rtt_nano_s31 (RT-Thread Nano v5.2.2, 无 IDF bootloader)
工具链	riscv32-esp-elf-gcc esp-15.2.0_20251204
链接参数	-march=rv32imafc_zicsr_zifencei -mabi=ilp32f -O2 -g -nostartfiles -ffunction-sections -fdata-sections
产物	build\app.elf (带调试信息) → build\app.bin (54656 字节, 烧到 flash 0x2000)
内存窗口	0x2F000000 .. 0x2F07AFC0 (491456 字节, S31 HP SRAM)

✦ 一个贯穿全文的提醒：**链接脚本是“内存地图”，启动文件是“进屋后先做什么”，ELF 是“这一切的物证”**。三者任何一处改错，症状往往都不在这三者身上（板子不启动、msh 起不来、跑一会儿复位……），所以学会“用 ELF 反查源码”是这项工作的核心技能。

0. 先建立直觉

0.1 flash 里躺着什么

flash 偏移	内容	谁写的
0x0000	保留 / 其它工程的东西	
0x2000	app.bin (54656 字节)	← esptool write-flash 0x2000 app.bin
	├ 头(32B): magic E9、4 段、	= ESP 镜像: 32 字节头 + 4 个段 + 校验/SHA
	├ entry=0x2F000000	
	├ 段0 → 0x2F000000 .boot	
	├ 段1 → 0x2F0000C0 .clic_entry	
	├ 段2 → 0x2F000140 .text	
	└ 段3 → 0x2F009840 .rodata+.rti_fn+.fsymtab+.data+.sdata	

这颗芯片上**没有 ID 的二级 bootloader**: bootROM 直接按镜像头把 4 个段搬进 RAM, 然后跳到 entry。 (0x2000 这个偏移就是 esptool 对 esp32s31 的 BOOTLOADER_FLASH_OFFSET, 见 README §7。)

0.2 RAM 里最终长什么样 (这是本文的主图)

左列是**真实地址**, 括号里是 nm / objdump 报出来的符号与大小:

0x2F000000	.boot	188 B (0xBC)	← 窗口底 = ORIGIN(RAM)
0x2F0000BC	4 B 空隙		_start (第 1 条指令)
0x2F0000C0	.clic_entry	66 B (0x42)	s31_clic_entry (64B 对齐)
0x2F000102	62 B 空隙 (ALIGN(64) 的代价)		
0x2F000140	.text	38624 B (0x96E0)	entry 在最前, SW_handler ...
0x2F009820	32 B 空隙		
0x2F009840	.rodata	15142 B (0x3B26)	字符串/常量 + 编译时间戳
0x2F00D366	2 B 空隙		
0x2F00D368	.rti_fn	40 B (0x28)	10 个函数指针 (自动初始化表)
0x2F00D390	.fsymtab	288 B (0x120)	24 条 msh 命令 (每条 12 B)
0x2F00D4B0	.data	200 B (0xC8)	有初值的全局量 (ROM 已搬好)
0x2F00D578	.sdata	12 B (0xC)	小组数据 (gp 相对寻址)
0x2F00D584	4 B 空隙		← 镜像内容到此为止 (PT_LOAD.filesz)
0x2F00D588	.bss	6456 B (0x1938)	startup.S 清零 (NOLOAD)
0x2F00EEC0	.sbss	144 B (0x90)	startup.S 清零 (NOLOAD)
0x2F00EF50	.noinit	16 B (0x10)	⚠ 不清零 (跨复位保留)
0x2F00EF60	heap ← __heap_start		RT-Thread 小内存堆 (≈ 40 KB)
	...		
0x2F077FC0	中断栈 4 KB ← __irq_stack_bottom		SW_handler 用
0x2F078FC0	启动栈 8 KB ← __rt_rvstack=__stack_bottom		startup.S 设 sp 用
0x2F07AFC0			← __stack_top = 窗口顶

记住这张图的三个“锚点”: **镜像只到 0x2F00D584** (.data/.sdata 的初值在里面), **BSS 三兄弟不进镜像** (NOLOAD), **栈从窗口顶向下、堆紧跟着镜像向上**。

0.3 从加电到 msh: 谁在哪一段代码里

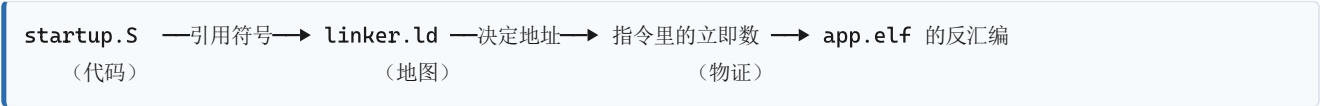
#	发生的事	在哪儿 (代码)	在哪儿 (地址/段)
1	bootROM 读 flash 0x2000 镜像头	芯片内部 ROM	—
2	按段表把 4 段搬进 RAM	芯片内部 ROM	写 0x2F000000/0C0/140/9840
3	跳到 entry = 0x2F000000	ROM	.boot
4	关 3 个看门狗 + 打开超级看门狗自动喂	startup.S	.boot
5	设 sp、gp	同上	.boot
6	关中断、设 mscratch、设 mtvec (CLIC)	同上	.boot
7	清 __bss_start..__bss_end	同上	清 .bss + .sbss
8	call entry → RT-Thread 的 C 入口	components.c	.text (第 1 个函数)
9	rt_hw_board_init(): 堆、"clock、SYSTIMER	board.c / drv_clk.c	.text
10	建 main/idle/timer 线程 → 启动调度器	components.c	.text
11	main 线程里跑 rt_components_init(): 遍历 .rti_fn	components.c	读 .rti_fn
12	INIT_DEVICE_EXPORT 的驱动注册 (USJ/GPIO/SPI/I2C) → finsh_system_init → msh 出来	各 drv_*.c	.rti_fn → .text

第 11 步是整篇文章的“枢纽”：它把「链接脚本里排好的一张表」变成「运行时真的被调用的函数」。第 4.5 节会把这张表的每 4 个字节都拆开给你看。

1. 三份“图纸”的分工

文件	它决定什么	什么时候生效	错了会怎样
bsp\startup.S	上电后头几十条汇编：关狗、sp、gp、mtvec、清 BSS、跳 C	运行时 (第 4~8 步)	板子反复复位 / gp 没设好 → 读写随机地址 / 中断进不来
bsp\linker.ld	每一段放在哪个地址、多长、什么对齐、谁在里面；并给出 __stack_top / __heap_start / __global_pointer\$ 等符号	链接时 (.o → .elf)	表和堆重叠 / 段没对齐 → 跳 0x8C / 栈撞镜像
build\app.elf	上述两者的物证：段表、符号表、机器码、调试信息	事后 (查问题时)	不会怎样——它是你唯一的“测谎仪”

一句话记住协作关系：



下一节开始就按这三步走：先看镜像怎么进来，再看 `_start` 每条指令对应的地址，最后逐段审链接脚本。

2. 第一站：镜像怎么进到 RAM（app.bin 的头、段、搬运）

2.1 头 32 字节，一个字段一个字段地读

偏移	00 01 02 03 04 05 06 07	08 09 0a 0b 0c 0d 0e 0f
0x00	e9 04 02 4f 00 00 00 2f	ee 00 00 00 20 00 00 00
0x10	00 ff ff 00 00 00 00 01	00 00 00 2f bc 00 00 00

偏移	字节	含义	在本镜像里
0	E9	镜像魔数	✓
1	04	段数	4
2	02	SPI 模式	DIO（-fm dio）
3	4F	高半字节 size / 低半字节 speed	16 MB / 80 MHz
4-7	00 00 00 2F	entry（小端）	0x2F000000 = <code>_start</code>
8	EE	wp_pin	0xEE = 未使用
9-11	00 00 00	spi_pin_drv	默认
12-13	20 00	chip_id	0x20 = 32（ESP32-S31）
14	00	min_chip_rev	0
15-16	00 00	min_chip_rev_full	0
17-18	FF FF	max_chip_rev_full	不限制
19-22	00 00 00 00	reserved	—
23	01	hash_appended	1 → 尾部有 32 字节摘要

💡 **entry 是链接脚本给的：** `linker.ld` 第 21 行 `ENTRY(_start)`，`_start` 落在 `.boot` 段首（`0x2F000000`）。所以“从哪开始执行”这件事，**不是 `startup.S` 决定的，是链接脚本决定的。**

2.2 段表：ROM 要搬 4 次

镜像头之后，每段 8 字节头（载入地址 + 长度）+ 数据：

#	载入地址	长度	文件范围	对应的段	说明
0	0x2F000000	0xBC (188)	[0x020,0x0DC)	.boot	正好一段
1	0x2F0000C0	0x44 (68)	[0x0E4,0x128)	.cltic_entry	段只有 0x42, 补到 4 字节对齐
2	0x2F000140	0x96E0 (38624)	[0x130,0x9810)	.text	—
3	0x2F009840	0x3D44 (15684)	[0x9818,0xD55C)	.rodata + .rti_fn + .fsymtab + .data + .sdata	五段连续, 合成一次搬运

三个关键结论:

1. 为什么是 4 段而不是 11 段: ELF 里其实只有 1 个可装载段 (见下)。esptool elf2image 把 ELF 里 8 个有内容的 section (.bss/.sbss/.noinit 是 NOLOAD, 不算) 按“数据末尾正好等于下一个段起点”的规则合并 (bin_image.py: merge_adjacent_segments()), 合并了 4 次 → 最后剩 4 个搬运段 (构建日志里那句 Merged 4 ELF sections. 就是这个意思)。合并的切点就是对齐空隙 (0xBC→0xC0、0x102→0x140、0x9820→0x9840), 而 0xD366→0xD368 那 2 字节没成为切点: .rodata 的数据被补齐到 4 字节 (0x3B26 → 0x3B28), 正好接上 .rti_fn 的起点 → 于是 .rodata + .rti_fn + .fsymtab + .data + .sdata 合并成一段 (0x3B28+0x28+0x120+0xC8+0xC = 0x3D44 ✓)。空隙本身不会被搬运 (少写 100 字节的 RAM), 但合段内部那 2 字节补齐会跟着走。
2. .bss/.sbss/.noinit 不在镜像里 (NOLOAD)。ELF 的 PT_LOAD 头把这件事写得很清楚:

```
Program Header:
  LOAD off 0x00001000 vaddr 0x2f000000 paddr 0x2f000000 align 2**12
        filesz 0x0000d584 memsz 0x0000ef60 flags rwx
```

filesz = 0xD584 正好等于 __sdata_end (镜像里真实有内容的字节数); memsz = 0xEF60 正好等于 __heap_start (0x2F00EF60)。memsz - filesz = 0x19DC = 6616 字节 = .bss + .sbss + .noinit, 即“占地址不占文件”的部分。

3. 段 3 里有 5 个段: 这就是为什么查问题时不要假设“一个 ELF 段 = 一次搬运”, 要么看 objdump -p (PT_LOAD), 要么读镜像头 (真实搬运表)。

2.3 这里不需要 AT>, 也不需要搬运 .data

ESP-IDF 工程里, .data 的运行地址 (VMA) 在 RAM、加载地址 (LMA) 在 flash, 所以链接脚本要写 AT> flash、启动代码要写一段“把 .data 从 flash 拷到 RAM”的循环。本工程完全不同:

	IDF 工程	本工程
镜像里有什么	bootloader + 应用 (.data 的初值以 LMA 存在 flash 侧)	只有 4 段, .data/.sdata 的初值按 VMA 排好
.data 谁搬	应用启动代码自己搬	bootROM 按段表搬 (段 3 里就带着)
linker.ld 要 AT> 吗	要	不要
startup.S 要搬 .data 吗	要	不要

代价与收益: 换来一个极简的 startup.S (188 字节), 代价是镜像就是 RAM 布局 (flash 占用 = RAM 镜像大小, 没有压缩、没有分页)。

2.4 bootROM 自己会打印它干了什么

captures\msh_accept1.txt（早期一版镜像的启动日志，注意那次是 5 段）：

```
rst:0x17 (CHIP_USB_UART_RESET),boot:0x58 (SPI_FAST_FLASH_BOOT)
...
load:0x2f000000,len:0x98          ← 段0（当时 .boot 只有 0x98 字节）
load:0x2f0000c0,len:0x44          ← 段1
load:0x2f000140,len:0x522c        ← 段2
load:0x2f005380,len:0x13c4        ← 段3
load:0x2f006748,len:0x8           ← 段4
entry 0x2f000000
```

⚠ 这段日志里的 rst:0x17 是 ROM 没有清掉的旧值（真的复位原因要读 LP_AONCLKRST_HPCORE0_RESET_CAUSE），这一条曾经把我们误导了一整轮——详见 README §4（复位原因表）。

这段日志最好的用法：它是“链接脚本 → 实际搬运”的免费核对表。load：的每一条都应该能在 objdump -h 的段表里找到对应的起始地址。

2.5 镜像尾部

```
0xd550: 78 d5 00 2f 78 d5 00 2f 00 36 6e 01 00 00 00 d4
0xd560: 30 be ae 95 e4 90 c2 78 0e 9e b4 f2 5c d2 4c 3b
0xd570: 00 78 71 31 fa c6 c3 1e e1 3c 7e 39 6a 88 48 af
          └────────── 32 字节摘要 (hash_appended=1) ─────────┘
```

- 0xD550..0xD55B 是段 3 的最后 12 字节，正好是 .sdata：78 d5 00 2f ×2 = 0x2F00D578 两个自指指针（_rt_thread_defunct 链表头初始化成指向自己），00 36 6e 01 = 0x016E3600 = 24 000 000（s_lcd_clk_hz，LCD 的 24 MHz）。→ .sdata 里到底有什么，可以直接从镜像里读出来，这就是“段 ↔ 数据”的对照快感。
- 0xD55C..0xD55F 这 4 个字节 = 3 字节 0 填充 + 1 字节校验和（实测值 00 00 00 d4），其后 32 字节是 SHA-256 摘要。两个都验过（python -m esptool --chip esp32s31 image_info 也报 Checksum: 0xd4 (valid) / Validation hash: 30be... (valid)）：

字段	算法 (esptool bin_image.py 实测对照)
校验和（1 字节，0xD55F）	0xEF 起，逐个异或全部 4 段的数据（不含任何头部）→ 0xD4
3 字节 0 填充	让“校验和”落在 16 字节整数倍的末尾（esptool 注释：checksum is stored in the last byte so that the file is a multiple of 16 bytes）
SHA-256（0xD560..0xD57F）	对它之前的整份文件（0x0000..0xD560，含头、段头、数据、填充、校验和）求 SHA-256——不是 ELF 的哈希

顺带一个实用结论：同一份源码两次编译，app.bin 也会有几十字节不同——.rodata 里有 __DATE__ / __TIME__ 组成的版本字符串（Sep 23 2026 / 23:54:48），再加上尾部摘要。比较两次构建是否“真的同一份代码”，要比 .text 段内容，不要比整个 app.bin。

2.6 同样叫 .bin：esptool elf2image 与 objcopy -O binary 的区别

这是最容易混淆的一对（“objdump 生成的 bin” 其实是 `objcopy -O binary`；`objdump` 只反汇编/转储，不产 bin）。两者的字节**逐段完全相同**，但用途完全不同：

	<code>objcopy -O binary app.elf plain.bin</code>	<code>esptool elf2image → app.bin</code>
本质	RAM 的平铺快照 ：从最低段地址 <code>0x2F000000</code> 连续铺到 <code>0x2F00D584</code>	给 bootROM 的搬运脚本 ：头 + 段表 + 数据 + 校验
头部	无	24 字节（magic <code>E9</code> / 段数 / flash 模式频率 / entry=0x2F000000 / 扩展头）
段表	无（地址隐含为“从 <code>0x2F000000</code> 开始”）	4 条（载入地址, len）+ 每段 8 字节头
对齐空隙（4 处共 100 B）	补 0 写进去	不写 （省传输，也省 flash）
段内补齐	无	+4 字节（ <code>.cllic_entry</code> 补到 4、 <code>.rodata</code> 补到 4 —— 顺带把 2 B 空隙吃掉）
校验	无	1 字节 XOR 校验和 + 32 字节 SHA-256
能不能直接烧到 <code>0x2000</code> 启动	✗ 没有 <code>E9</code> 头，ROM 不认	✓ 就是干这个的
适合用来	和 RAM 布局逐字节对照、做 diff、喂给别的工具	烧录； <code>image_info</code> 还能验校验和

尺寸账（同一次构建，实测）：

成分	<code>objcopy</code> 平铺	<code>esptool</code> 镜像
真实段数据（8 个 PROGBITS 段之和）	54560	54560
段内补齐到 4 字节	0	+4
4 处对齐空隙	+100（补 0）	0
24 B 基头 + 4×8 B 段头	0	+56
校验和 1 B + 填充 3 B	0	+4
SHA-256 摘要	0	+32
合计	54660 （ <code>0xD584</code> ）	54656 （ <code>0xD580</code> ）

两者只差 4 字节纯属巧合（100 vs 96）。别用它俩的“大小差不多”推断“内容差不多”：平铺 bin 里第 0 个字节是 `.boot` 的第一条指令，而 `esptool` 镜像里第 0 个字节是魔数 `0xE9`。

更实用的一条：`-fm/-ff/-fs` 这几个参数**只改头部**（实测把它们从 `-fm dio -ff 80m -fs 16MB` 换成 `-fm qio -ff 40m -fs 8MB`：整份镜像只有 **34 字节不同** = 2 个头字节 + 32 字节摘要，四段数据逐字节不变）。所以“烧录参数”不会影响程序本身，只会影响 ROM 怎么配置 flash —— 以及**摘要**（因为摘要覆盖头部）。

3. 第二站：_start 逐条对照（源码 ↔ 反汇编 ↔ 为什么）

`bsp\startup.S` 一共 105 行，编译出来 **188 字节**（`0xBC` / 47 条指令，含压缩指令）。下面按源码顺序分块对照。反汇编用：

```
$od -d --start-address=0x2f000000 --stop-address=0x2f0000c0 build\app.elf
```

3.1 只让 core0 干活 + 关三个看门狗（源码 31–49 行）

源码	反汇编	为什么
csrr t0, mhartid	2f000000: csrr t0,mhartid	S31 是多 hart, ROM 可能让每个 hart 都跳到这里
bnez t0, .Lpark	2f000004: bnez t0,2f0000b6	非 0 号 hart 直接去 wfi 循环, 不执行任何初始化 (避免两个核同时清 BSS)
li t1, WDT_WKEY	lui t1,0x50d84 + addi t1,t1,-1375 → # 50d83aa1 <WDT_WKEY>	32 位常量 (0x50D83AA1) 装不进 12 位立即数 → li 变两条指令。 objdump 把结果标成 .set 常量名 WDT_WKEY (它在符号表里是 *ABS* 绝对符号)
sw t1, RTC_WDT_WPROTECT(t0)	sw t1,24(t0) # 20801018	0x18 就是源码里的偏移常量; lui t0,0x20801 提供高 20 位
sw zero, RTC_WDT_CONFIG0(t0)	sw zero,0(t0)	关 RTC 看门狗
sw zero, RTC_WDT_WPROTECT(t0)	sw zero,24(t0)	上锁 (写回非 key 值即可锁)
TIMG0 三条	lui t0,0x20580 + sw t1,100(t0) / sw zero,72(t0) / sw zero,100(t0)	0x64 = WDTWPROTECT、0x48 = WDTCONFIG0; 实测 MWDT 会在 1 秒后复位芯片 , 所以必须关
TIMG1 三条	lui t0,0x20581 + 同样三条	第二个定时器组, 同样必须关

3.2 喂超级看门狗（源码 51–65 行）

源码	反汇编	为什么
解锁 SWD_WPROT	lui t0,0x20801 + lui t1,0x50d84 / addi t1,t1,-1375 + sw t1,32(t0)	偏移 0x20
读改写 SWD_CONF 的 bit18	lw t2,28(t0) → lui t3,0x40 → or t2,t2,t3 → sw t2,28(t0)	1<<18 = 0x40000, lui t3,0x40 正好一条装好; 这是标准的 read-modify-write
上锁	sw zero,32(t0)	偏移 0x1C = SWD_CONFIG

🔥 这是本工程最贵的一课 (README §4 有完整记录): 我们关了 RTC_WDT, 但**超级看门狗盯的是 RTC_WDT 的喂狗信号**——没人喂 → 每 ~3.4 秒整片复位一次。现象是“串口只看到开机几行就重来”, 而 ROM 打印的 rst:0x17 (USB 复位) 是**没清掉的旧值**, 把排查带偏了一整轮。现在的做法照抄 IDF 的 bootloader_super_wdt_auto_feed(): 开 SWD_AUTO_FEED_EN。

3.3 栈指针与 gp（源码 67–77 行）

源码	反汇编	为什么
<code>la sp, __stack_top</code>	2f000064: auipc sp,0x7b + 2f000068: addi sp,sp,-164 # 2f07afc0 <__stack_top>	<code>la</code> = PC 相对两条; 结果是 内存窗口顶端 (栈向下长)
<code>.option norelax + la gp, __global_pointer\$</code>	2f00006c: auipc gp,0xe + 2f000070: addi gp,gp,-756 # 2f00dd78	没有 <code>crt0</code> , <code>gp</code> 只能自己装; <code>norelax</code> 防止这条 <code>la</code> 自己被松弛成 <code>gp</code> 相对 (鸡生蛋)
<code>.option pop</code>	—	之后的 <code>la</code> 又可以被松弛了 (见 3.5 的 <code>addi t0,gp,-2032</code>)

gp 是什么、为什么必须有: RISC-V 的 ABI 约定“小数据区”用 `gp` 相对寻址 (`lw a5,292(gp)` 一条指令, 而不是 `lui+addi+lw` 三条)。GCC 默认把 ≤ 8 字节的静态变量放进 `.sdata/.sbss`。没有 `crt0` 就必须自己保证 `gp` 指向小数据区中部, 否则那些访问会打到随机地址上 (症状: 一进 `rt_tick` 相关代码就跑飞)。详见第 6 节的实测数据。

3.4 关中断、中断栈、CLIC 陷阱入口 (源码 79–89 行)

源码	反汇编	为什么
<code>csrci mstatus, 0x8</code>	2f000074: csrci mstatus,8	bit3 = MIE, 清 0 = 关全局中断 (初始化期间不想被打断)
<code>li t0,0x6000 + csrs mstatus,t0</code>	<code>lui t0,0x6 + csrs mstatus,t0</code>	置 FS=3 (FPU 打开) : - <code>mabi=ilp32f</code> 下陷阱保存要写 <code>f0-f31</code> , <code>FS=0</code> 时那几条会 二次陷阱 (踩过)
<code>la t0, __rt_rvstack + csrwr mscratch,t0</code>	<code>auipc t0,0x79 + addi t0,t0,-190 # 2f078fc0 + csrwr mscratch,t0</code>	<code>SW_handler</code> 第一件事就是 <code>csrrwr sp,mscratch,sp</code> —— 线程栈 → 中断栈 的切换全靠这个寄存器
<code>la t0, s31_clic_entry + ori t0,t0,3 + csrwr mtvec,t0</code>	<code>auipc + addi # 2f0000c0 <s31_clic_entry> + ori t0,t0,3 + csrwr mtvec,t0</code>	<code>mtvec = 0x2F0000C3</code> : 低位 2 位 = 3 = CLIC 模式 , 高 30 位的基址硬件只认 [31:6], 所以入口必须 64 字节对齐 (第 4.2 节)

3.5 清 BSS (源码 91–98 行) —— 全篇最值得看的一段

源码	反汇编	为什么
<code>la t0, __bss_start</code>	2f00009a: addi t0,gp,-2032 # 2f00d588 <idle_thread_stack>	★ 两条指令被链接器松弛成一条 : <code>__bss_start = 0x2F00D588</code> , 而 <code>gp = 0x2F00DD78</code> , 差 <code>-2032</code> , 塞得进 12 位立即数 → 直接 <code>addi t0,gp,-2032</code> (省 4 字节)
<code>la t1, __bss_end</code>	2f00009e: auipc t1,0xf + 2f0000a2: addi t1,t1,-334 # 2f00ef50 <s_clk_guard_skips_inv>	距 <code>gp</code> 有 <code>0x1178 (4472)</code> 字节 → 超出 ± 2 KB , 松弛不了, 只能两条 PC 相对指令
<code>bgeu t0,t1,2f</code>	2f0000a6: bgeu t0,t1,2f0000b4	空区间保护 (两个符号相等时直接跳过)
<code>sw zero,0(t0) / addi t0,t0,4 / bltu t0,t1,1b</code>	2f0000aa / ae / b0	3 条指令的循环, 字对齐写 0; 范围是 <code>.bss + .sbss</code> (6616 字节, 约 1654 次循环)

★ **注意 objdump 的显示陷阱**：它显示的是 `idle_thread_stack`、`s_clk_guard_skips_inv`，而源码里写的是 `__bss_start`、`__bss_end`。原因是**同一地址上有多个符号**，反汇编器只挑一个显示：

地址	同一地址上的符号	类型
0x2F00D588	<code>__bss_start</code> (<code>linker.ld</code> 第 89 行)、 <code>idle_thread_stack</code> (<code>idle.c</code> 里的静态数组)	前者是“段符号”，后者是“变量符号”
0x2F00EF50	<code>__bss_end</code> (第 119 行 <code>PROVIDE</code>)、 <code>__noinit_start</code> (第 107 行)、 <code>__sbss_end</code> (第 100 行)、 <code>s_clk_guard_skips_inv</code> (<code>drv_clk.c</code> 的 <code>.noinit</code> 变量)	四个名字，一个地址

这正是 `.noinit` 那件事的关键：`__bss_end` 与 `__noinit_start` 是**同一个地址**，意味着“清零循环的终点”和“跨复位保留区的起点”严丝合缝——多清一个字节就会把哨兵清掉（第 8.3 节复盘）。要确认“哪个名字对应哪个地址”，用 `nm -n` 或 `build/app.map`：

```
2f00ef50 B __bss_end          ← nm 里 __bss_end 也在这个地址（与 __noinit_start 同址）
2f00ef50 b s_clk_guard_skips_inv
```

3.6 进 RT-Thread（源码 99–103 行）

源码	反汇编	为什么
<code>call entry</code>	<code>2f0000b4: jal 2f000140 <entry></code>	<code>call</code> 是伪指令（ <code>auipc+jalr</code> ），链接器发现目标在 ± 1 MB 内 → 松弛成一条 <code>jal</code> （省 4 字节）。目标 <code>0x2F000140</code> 就是 <code>.text</code> 的第一个函数
<code>.Lpark: wfi / j .Lpark</code>	<code>2f0000b6: wfi + 2f0000ba: j 2f0000b6</code>	非 0 号 hart 的停靠点（ <code>bnez</code> 的目标）。 注意 <code>wfi</code> 在中断被关掉时只会等中断，不保证停机 ，所以外面套一个死循环

3.7 源码里的符号 ↔ ELF 里的值（一张对得上的表）

startup.S 里写的	ELF 里的值	谁给的	linker.ld 行
__stack_top	0x2F07AFC0	ORIGIN(RAM)+LENGTH(RAM)	137
__global_pointer\$	0x2F00DD78	MIN(__sdata_start+0x800, ...)	129-130
__rt_rvstack	0x2F078FC0	__stack_bottom (__stack_top-0x2000)	138-139
s31_clic_entry	0x2F0000C0	trap_gcc.S, 被 .cllic_entry 段收走 (ALIGN(64))	35-38
__bss_start	0x2F00D588	.bss 段开头	89
__bss_end	0x2F00EF50	PROVIDE(__bss_end = __sbss_end)	119
entry	0x2F000140	components.c, 被 .text 段收走且排在最前	41-44

★ 这些符号**不占内存**，它们只是“地址的别名”（nm 里显示为 A/绝对符号，或者跟着所在段显示为 B）。链接脚本用 PROVIDE(...) 写，意思是“如果没人定义它，就定义成这个值”。

3.8 startup.S 故意没做的事（以及为什么可以不做）

没做的事	为什么不需要
搬 .data / .sdata 初值	ROM 按 VMA 搬好了（第 2.3 节）
初始化 __libc_init_array / C++ 构造	纯 C 工程 + -nostartfiles
关 cache / 打开 cache	ROM 已经配好；本工程全程不碰 cache 控制寄存器
配置 PMP / 其它 CSR	ROM 已配；不跑用户态
清 .noinit	故意不清：它是跨复位的哨兵（提频守卫）
设 mstatus.FS	✗ 其实 必须做 ，而且做了（3.4 节）；漏了会二次陷阱
设 mscratch	✗ 必须做，SW_handler 靠它换栈

4. 第三站：linker.ld 逐段对照

4.0 段总表（这份表就是“内存地图”本身）

```
$od -h build\app.elf      # 或 riscv32-esp-elf-objdump -h
```

#	段	起始地址	大小	结束（下一个地址）	对齐	在镜像里	谁贡献
0	.boot	0x2F000000	0xBC (188)	0x2F0000BC	4	✓	startup.o (_start)
1	.clic_entry	0x2F0000C0	0x42 (66)	0x2F000102	64	✓	trap_gcc.o (s31_clic_entry)
2	.text	0x2F000140	0x96E0 (38624)	0x2F009820	64	✓	所有 .o 的代码 (components.o 的 entry 在最前)
3	.rodata	0x2F009840	0x3B26 (15142)	0x2F00D366	64	✓	只读数据/字符串
4	.rti_fn	0x2F00D368	0x28 (40)	0x2F00D390	4	✓	components.o / shell.o / 各 drv_*.o / drv_usj_dev.o
5	.fsymtab	0x2F00D390	0x120 (288)	0x2F00D4B0	4	✓	msh 命令表 (5 个 .o)
6	.data	0x2F00D4B0	0xC8 (200)	0x2F00D578	8	✓	object.o
7	.sdata	0x2F00D578	0xC (12)	0x2F00D584	8	✓	defunct.o + drv_lcd_axs15352.o
8	.bss	0x2F00D588	0x1938 (6456)	0x2F00EEC0	8	X NOLOAD	全部 .o 的无初值全局量
9	.sbss	0x2F00EEC0	0x90 (144)	0x2F00EF50	8	X NOLOAD	≤8 字节静态量 (内核热点全在这)
10	.noinit	0x2F00EF50	0x10 (16)	0x2F00EF60	8	X NOLOAD	drv_clk.o (提频守卫)

空隙总账（对齐的代价，一共 100 字节，且不进镜像）：

空隙	大小	谁要求的
0xBC → 0xC0	4	.clic_entry 的 ALIGN(64)
0x102 → 0x140	62	.text 的 ALIGN(64)
0x9820 → 0x9840	32	.rodata 的 ALIGN(64)
0xD366 → 0xD368	2	.rti_fn 的 ALIGN(4)

为什么可以放心留空隙：ROM 按段搬运，空隙不占 flash 也不占传输时间；而 64 字节对齐换来的好处（CLIC 入口合法性）远大于 100 字节的地址浪费。

4.1 .boot：ROM 跳进来的第一站

```
.boot : { KEEP(*(.boot)) } > RAM
```

- 为什么**必须**在 0x2F000000：镜像头的 entry 指向 _start，而 _start 在 .boot 段；段从 ORIGIN(RAM) 开始排，所以第一段就是它。这是“约定”而不是“硬件要求”——只要 ENTRY()/_start 一致，放别处也行；但放最

前面最直观，也让 `entry` 地址 = RAM 窗口底。

- `KEEP(...)` 在这里其实是“防御性”的：`ENTRY(_start)` 已经把 `_start` 变成 GC 根，但显式 `KEEP` 让“如果哪天有人改了 `ENTRY()`”也不会把启动代码删掉。

4.2 `.cllic_entry`：64 字节对齐的跳板

```
.cllic_entry : { . = ALIGN(64); KEEP(*(.cllic_entry)) } > RAM
```

- 内容是 `trap_gcc.S` 里的：

```
.section .cllic_entry, "ax"
.align 6                      /* 2^6 = 64 */
.globl s31_cllic_entry
s31_cllic_entry:
    j SW_handler
```

- 反汇编（真实）：

```
2f0000c0 <s31_cllic_entry>:
2f0000c0: 0d00006f j 2f000190 <SW_handler>    ← 整个跳板就这 4 字节
2f0000c4: 0001      nop                ← 其余 62 字节全是 nop 填充
... (14 条 nop)
```

- **为什么要 64 字节**：CLIC 模式下 `mtvec` 的低 2 位是模式 (3)，`mtvec[31:6]` 才是处理函数基址，硬件会把低 6 位当 0。所以入口地址不是 64 的倍数 → 实际跳到一个错误的地方（症状极隐蔽：中断一进来就跑飞）。`linker.ld` 第 146 行有一条 `ASSERT` 兜底：

```
ASSERT((s31_cllic_entry & 0x3F) == 0, "s31_cllic_entry MUST be 64-byte aligned for CLIC mtvec")
```

另外 `build.ps1` 每次链接后还会打印一行 `CLIC entry @ 0x2F0000C0 64B aligned: True` —— **两道保险**。

- 跳板为什么要有：`mtvec` 只能指一个地址，而真正的处理函数 `SW_handler` 在 `.text` 里（由 `libcpu` 的汇编实现，位置不受我们控制）。跳板把这层间接固定下来。

4.3 `.text`：为什么 `ALIGN(64)`、为什么 `.text.entry` 在最前

```
.text : ALIGN(64) {
    KEEP(*(.text.entry))
    *(.text*)
} > RAM
```

- `ALIGN(64)`：让代码段起点也落在 64 字节边界（好习惯，便于将来做 CLIC 向量化/对齐优化），代价就是上面那 62 字节空隙。
- `KEEP(*(.text.entry))` 放最前：`entry()`（`rt-thread/src/components.c`）是 C 世界的入口，`startup.S` 用 `call entry` 跳到它。它必须排在 `.text` 最前面的原因是 **它没有任何“被引用”的证据**（唯一的引用来自汇编里的 `call entry` —— 那也算引用，但把 `.text.entry` 单独做一段 + `KEEP` + 放最前，可以保证：① 不会被 `--gc-sections` 删；② 地址稳定（固定在 `0x2F000140`，方便对照与断言）。
- 真实反汇编的第一屏（`entry` 把 RT-Thread 的启动顺序全暴露了）：

```

2f000140 <entry>:
2f000140: addi sp,sp,-16
2f000142: sw    ra,12(sp)
2f000144: jal   2f0032ba <rt_hw_interrupt_disable>
2f000148: jal   2f005b8c <rt_hw_board_init>          ← 堆 + 时钟 + SYSTIMER
2f00014c: jal   2f001202 <rt_show_version>
2f000150: jal   2f002762 <rt_system_timer_init>
2f000154: jal   2f001b08 <rt_system_scheduler_init>
2f000158: lui   a1,0x2f000
2f000166: addi  a1,a1,862    # 2f00035e <main_thread_entry> ← 函数指针当参数传给 rt_thread_create
2f000170: addi  a0,a0,-1984 # 2f009840 ("main" 字符串, 在 .rodata)
2f000174: jal   2f0020b2 <rt_thread_create>
2f000178: jal   2f002282 <rt_thread_startup>
2f00017c: jal   2f00277a <rt_system_timer_thread_init>
2f000180: jal   2f0004e0 <rt_thread_idle_init>
2f000182: jal   2f0004ce <rt_thread_defunct_init>
2f000184: jal   2f001c3a <rt_system_scheduler_start>

```

🔗 两个可以直接拿去用的技巧：① `lui a1,0x2f000 + addi a1,a1,862` 是“把函数地址装进参数寄存器”（R_RISCV_HI20/L012_I 重定位），objdump 的注释 # 2f00035e <main_thread_entry> 就是链接器填好的结果；② `addi a0,a0,-1984 # 2f009840` 指向 .rodata 里的 "main" 字符串——**objdump 显示不出字符串内容**，要自己回 .rodata 里看（这解释了为什么“看反汇编看不出是哪个字符串”）。

4.4 .rodata：15 KB 的只读数据

```
.rodata : ALIGN(8) { *(.rodata*) *(.srodata*) } > RAM
```

- `*(.srodata*)` 也要收：GCC 会把“小只读数据”单独放 .srodata，忘了收就会在链接时报未定义/丢数据。
- 这里面除了字符串常量，还有**编译时间戳**（`__DATE__` / `__TIME__`，被 `rt_show_version()` 打印）：

```
2f009860: "- RT - Thread Operating System\n" "23:54:48" "\0" "Sep 23 2026"
```

→ 所以**同一份源码两次编译，ELF 不是逐字节相同的**（第 2.5 节）。

- .rodata 在 RAM 里（不是 XIP）：本工程把整个镜像搬进 RAM，读只读数据就是普通内存访问（比 IDF 那种“代码从 flash 取指”要快，代价是占 RAM）。

4.5 ★ .rti_fn：RT-Thread 自动初始化表（本文最该看懂的一段）

```

.rti_fn : ALIGN(4) {
    __rti_fn_start = .;
    KEEP(*(SORT(.rti_fn*)));
    __rti_fn_end = .;
} > RAM

```

① 表里的原始字节（`objdump -s -j .rti_fn`）

```
2f00d368 5a03002f 9003002f 9403002f 5a5e002f
2f00d378 bc5f002f 8469002f 546e002f 007b002f
2f00d388 2634002f 9803002f
```

每 4 字节是一个函数指针（小端）。逐条解码（值 → 符号名）：

槽位	段名（map 里）	存的值	指向的函数	谁生成的	级别（宏）
2f00d368	.rti_fn.0	2f00035a	rti_start	components.c	INIT_EXPORT(rti_start,"0") — 空函数
2f00d36c	.rti_fn.0.end	2f000390	rti_board_start	components.c	"0.end" — 空函数
2f00d370	.rti_fn.1.end	2f000394	rti_board_end	components.c	"1.end" — 空函数
2f00d374	.rti_fn.3	2f005e5a	s31_usj_dev_register	drv_usj_dev.c	INIT_DEVICE_EXPORT
2f00d378	.rti_fn.3	2f005fbc	s31_systick_start	drv_systick.c	INIT_DEVICE_EXPORT
2f00d37c	.rti_fn.3	2f006984	s31_gpio_init	drv_gpio.c	INIT_DEVICE_EXPORT
2f00d380	.rti_fn.3	2f006e54	s31_spi_init	drv_spi.c	INIT_DEVICE_EXPORT
2f00d384	.rti_fn.3	2f007b00	s31_i2c_init	drv_i2c.c	INIT_DEVICE_EXPORT
2f00d388	.rti_fn.6	2f003426	finsh_system_init	shell.c	INIT_APP_EXPORT
2f00d38c	.rti_fn.6.end	2f000398	rti_end	components.c	"6.end" — 空函数

② 这些槽位是谁、怎么产生的

rtdef.h 里那行宏（本配置下走的是最简那条）：

```
#define INIT_EXPORT(fn, level) \  
    rt_used const init_fn_t __rt_init_##fn rt_section(".rti_fn." level) = fn  
#define INIT_DEVICE_EXPORT(fn)  INIT_EXPORT(fn, "3")  
#define INIT_APP_EXPORT(fn)     INIT_EXPORT(fn, "6")
```

翻译过来：“定义一个常量函数指针，放进名为 .rti_fn.3 的段里”。所以 INIT_DEVICE_EXPORT(s31_spi_init) 干了三件事：① 生成符号 __rt_init_s31_spi_init；② 它的值 = s31_spi_init 的地址；③ 它落在 .rti_fn.3 段 → 被链接脚本收进 .rti_fn 的对应位置。

③ SORT(...) 为什么能保证“先板级、后设备、再应用”

SORT_BY_NAME(.rti_fn*) 按段名字符串排序，而名字就是级别：

```
.rti_fn.0 < .rti_fn.0.end < .rti_fn.1.end < .rti_fn.3 < .rti_fn.6 < .rti_fn.6.end
```

ASCII 逐字符比较：'0' < '1' < '3' < '6'，前缀短的排前面 —— 于是数字级别 = 执行顺序。（同一个级别里多个函数之间的顺序不保证，所以“有依赖关系的初始化”不要放同一级。）

④ 谁在跑这张表（真实反汇编）

components.c 第 126 行：

```
for (fn_ptr = &__rt_init_rti_board_end; fn_ptr < &__rt_init_rti_end; fn_ptr++) (*fn_ptr)();
```

对应的机器码（`main_thread_entry` 里，`rt_components_init()` 被内联）：

```
2f00035e <main_thread_entry>:
2f000364: lui    s0,0x2f00d
2f000368: lui    s1,0x2f00d
2f00036e: addi s0,s0,880    # 2f00d370 <__rt_init_rti_board_end>    ← 起点
2f000372: addi s1,s1,908    # 2f00d38c <__rt_init_rti_end>          ← 终点
2f000376: bgeu s0,s1,2f000384
2f00037a: lw     a5,0(s0)    ← 取一个函数指针
2f00037c: addi s0,s0,4       ← 下一个槽位
2f00037e: jalr a5            ← 调用它
2f000380: bltu s0,s1,2f00037a
2f000384: ...               ← 之后 tail-call 到 main
2f00038c: j      2f008e3c <main>
```

这段代码把三件事一次性讲清楚了：

1. 两个立即数就是表的地址：0x2F00D370（起点）与 0x2F00D38C（终点）。起点是 `rti_board_end` 槽位本身（不是 +4），所以第一次“调用”的是边界空函数 `rti_board_end()`（反汇编是 `li a0,0; ret`）——**无害且是设计的一部分**：边界符号同时充当“空函数”和“区间端点”。
2. KEEP 为什么必须：**没有任何指令引用 0x2F00D374..0x2F00D388 这些槽位**，它们只是被“按地址遍历”。链接器 `--gc-sections` 看不到引用 → 会把整张表当垃圾删掉。症状极具欺骗性：**版本号、main、调度器全都正常，只是所有 `INIT_*_EXPORT` 的函数一个都不跑**（tick 不开、msh 不出现）。这是本工程踩过的坑之一（第 8.2 节）。
3. 表遍历是按地址区间的，所以表的物理顺序 = 执行顺序，这也是为什么 `.rti_fn` 必须放在 `.bss` 之前（否则会被堆写花，见 4.8 与第 8.1 节）。

⑤ 一个本 port 的实测发现（诚实记录）

`nm` 里没有 `rt_components_board_init` 这个符号，反汇编里也没有任何指令引用 0x2F00D368（`rti_start`）与 0x2F00D36C（`rti_board_start`）。结论：**本 port 里 0/1 级（板级）初始化这半张表根本没被遍历**——因为 `board.c` 没有调用 `rt_components_board_init()`，板级初始化是我们自己在 `rt_hw_board_init()` 里显式做的。而 0/1 级槽里只有两个空函数，所以“不跑”没有丢任何功能。

⚠ **推论（重要）**：在本 port 里写 `INIT_BOARD_EXPORT(fn)` 的 `fn` **永远不会被执行**。要参与自动初始化就用 `INIT_DEVICE_EXPORT`（3 级）或更高级别。`drv_systick.c` / `drv_gpio.c` 的注释里已经记着这条（当时就是被它坑过）。

4.6 .fsymtab：msh 命令表（和 .rti_fn 同一类技巧）

```
.fsymtab : ALIGN(4) {
    __fsymtab_start = .;
    KEEP(*(FSymTab));
    __fsymtab_end = .;
} > RAM
```

- `MSH_CMD_EXPORT_ALIAS(lcd_msh, lcd, "...")` 展开成（`finsh.h`）：


```
rt_used const struct finsh_syscall __fsym_lcd rt_section("FSymTab") = {
    "lcd",          /* name */
    "...描述...",   /* desc */
    (syscall_func)&lcd_msh
};
```

`struct finsh_syscall` = 3 个指针 = **12 字节**，和 `.rti_fn`（4 字节/条）一样是“靠段名收集”。

- **账能对上**：0x120 = 288 = 24 × 12，而 map 文件里正好是

贡献者	条数	大小
msh.o (free/ps/help)	3	0x24
cmd.o (list/version/clear)	3	0x24
dev_pin.o (pin)	1	0xC
drv_lcd_axs15352.o (lcd)	1	0xC
main.o (s31_info...reboot 等 16 条)	16	0xC0
合计	24	0x120 ✓

- 真实前 4 条（每个字段的指针都能回查到字符串与函数）：

槽位	name → 字符串	desc → 字符串	func
2f00d390	2f00c528 = "free"	2f00c504 = "Show the memory usage in the system"	2f003c58 = cmd_free
2f00d39c	2f00c54c = "ps"	2f00c530 = "List threads in the system"	2f003c4a = cmd_ps
2f00d3a8	2f00c560 = "help"	2f00c550 = "RT-Thread shell ..."	2f003bea = msh_help
2f00d3b4	2f00c580 = "list"	2f00c570 = "list objects"	2f004c38 = cmd_list

这就是“**msh 里能敲到哪些命令**”的物理来源：不是注册函数调用出来的，而是**链接器把 24 条 12 字节的记录排在了 0x2F00D390..0x2F00D4B0**，`finsh_system_init()` 遍历 `__fsymtab_start..__fsymtab_end` 建索引。想确认某条命令在不在镜像里，直接 `objdump -s -j .fsymtab + nm` 就能查。

4.7 .data 与 .sdata：带初值的全局量

```
.data : ALIGN(8) { *(.data*) } > RAM
.sdata : ALIGN(8) { __sdata_start = .; *(.sdata*) __sdata_end = .; } > RAM
```

- **.data = 200 字节，全都来自 object.o**（`.data._object_container`），内容实测：

```
2f00d4b0 01000000 b4d4002f b4d4002f 9c000000
2f00d4c0 00000000 02000000 c8d4002f c8d4002f
2f00d4d0 28000000 00000000 03000000 dcd4002f
```

按 20 字节一条正好 **10 条**（类型 1..10 = RT-Thread 的 10 类内核对象）： 每条 = 类型号 + 两个**指向自己的链表指针**（空链表头的标准写法）+ 对象大小。例：第 1 条类型=1（线程），0x9C = 156 = sizeof(struct rt_thread)。→ .data 为什么在镜像里：这些初值必须真的存在 RAM 里，而 ROM 已经把它们搬好了（第 2.3 节）。

- `.sdata` = 12 字节, 2 个变量 (`objdump -s -j .sdata` 可以逐字节看) :

变量	地址/内容	说明
<code>_rt_thread_defunct</code> (<code>defunct.o</code>)	0x2F00D578, 值 = 0x2F00D578 两个	回收线程链表的空头 (自指)
<code>s_lcd_clk_hz</code> (<code>drv_lcd_axs15352.o</code>)	0x2F00D580, 值 = 0x016E3600 = 24 000 000	LCD SPI 时钟 24 MHz

为什么只有 12 字节： `.sdata` 只收“GCC 认为够小（默认 ≤ 8 字节）且有初值”的静态量；`_rt_thread_defunct` 是 8 字节指针 $\checkmark$ ，`s_lcd_clk_hz` 是 4 字节 `rt_uint32_t` $\checkmark$ 。其余零散小量都进了 `.sbss`（无初值版）。

4.8 .bss 与 .sbss：RAM 都花在哪儿了

```
.bss (NOLOAD) : ALIGN(8) { __bss_start = .; *(.bss*) *(COMMON) . = ALIGN(8); } > RAM
.sbss (NOLOAD) : ALIGN(8) { __sbss_start = .; *(.sbss*) *(.scommon*) . = ALIGN(8); __sbss_end = .; }
> RAM
```

(1) .bss 的构成 (6456 字节, 从 map 里抄出来的明细)

地址	符号	大小	谁
2F00D588	idle_thread_stack	0x200 (512)	idle.o
2F00D788	idle_thread	0x9C	idle.o
2F00D824	rt_log_buf.0	0x80	kservice.o
2F00D8A4	_lock	0x34	kservice.o
2F00D8D8	rt_thread_priority_table	0x100	scheduler_up.o
2F00D9D8	_timer_thread_stack	0x400 (1024)	timer.o
2F00DD8	_soft_timer_sem / _timer_thread	0x28 + 0x9C	timer.o
2F00DE9C	finsh_prompt.0	0x81	shell.o
2F00DF20	_hw_pin	0x50	dev_pin.o
2F00DF70	s_tx_ring	0x800 (2048)	drv_usj.o (控制台发送环)
2F00E770	s_usj_dev + s_rx_ring	0x4C + 0x200	drv_usj_dev.o
2F00E9BC	s31_isr_table	0x100 (64 项 × 4)	trap_handler.o (CLIC 分发用)
2F00EABC	s_qflash_dev / s_flash_dev / rx / tx	0x78 / 0x60 / 0x40 / 0x40	drv_spi.o
2F00EC14	s_qspi2_bus / s_spi2_bus	0x8C × 2	drv_spi.o
2F00ED2C	s_i2c1 / s_i2c0	0x98 × 2	drv_i2c.o
2F00EE5C	s_lcd_dev	0x60	drv_lcd_axs15352.o
—	两次 *fill* (对齐填充)	3 + 4	链接器
	合计	0x1938 ✓	

🔗 光看这张表就能回答“RAM 为什么不够用”：SPI/LCD/I2C 的驱动结构体 0x60~0x98，控制台环形缓冲 0x800+0x200，如果再加一个大缓冲（比如 240×296×2 = 142 KB 的整屏帧缓冲），就必须放进 PSRAM 或从堆里分配。

(2) .sbss 的构成 (144 字节 / 32 个符号)

GCC 把小静态量塞这里，RT-Thread 最热的变量全在：

```
rt_tick rt_interrupt_nest rt_current_priority rt_thread_ready_priority_group
rt_scheduler_lock_nest rt_thread_switch_interrupt_flag rt_interrupt_from/to_thread
system_heap _console_device _soft_timer_list _timer_list shell __rt_errno
s_clk_guard_armed s_lcd_ready s_rx/tx_head/tail ...
```

(3) 为什么 .bss 夹在 .sdata 和 .sbss 中间 (一个重要副作用)

链接脚本的字面顺序是 .data → .sdata → .bss → .sbss，于是两个“小数据段”被 6456 字节的 .bss 隔开了：

.sdata 在 0x2F00D578，.sbss 在 0x2F00EEC0，相距 0x1948 (6472) 字节。一个 gp 只能覆盖 ±2 KB，所以两者不可能同时被覆盖——这正好是第 6 节那个实验的由来。

(4) __bss_start..__bss_end 为什么必须是“两段之和”

.bss 和 .sbss 相邻, 且 PROVIDE(__bss_end = __sbss_end) 让终点落在 .sbss 末尾。于是 startup.S 里那个循环 (__bss_start...__bss_end) 就正好清掉**两段、且只清这两段**。如果改成 .sbss → .bss 的顺序, 就必须同时把 __bss_end 的定义挪到 .bss 末尾 (第 6 节的实验 A 就是这么改的)。

4.9 .noinit：唯一“不清零”的 16 字节

```
.noinit (NOLOAD) : ALIGN(8) {
    __noinit_start = .; KEEP(*(.noinit)) KEEP(*(.noinit*)) . = ALIGN(8); __noinit_end = .;
} > RAM
```

- 内容 (nm 实测) 就是 drv_clk.c 里提频守卫的 3 个 rt_uint32_t：

地址	符号	作用
2F00EF50	s_clk_guard_skips_inv	“连续跳过次数”的反码 (自校验用)
2F00EF54	s_clk_guard_skips	连续跳过次数
2F00EF58	s_clk_guard	哨兵：0x5A5A5A5A 表示“上次提频后没跑到确认”

- 三个符号的地址关系解释了**为什么它能跨复位活下来**：bootROM 不清 SRAM，startup.S 只清 __bss_start...__bss_end (正好停在 0x2F00EF50)，堆又从 __noinit_end = 0x2F00EF60 开始。**上不挨清零、下不挨堆**——位置是被“夹”出来的，不是随便放的。
- ⚠ 这个段的位置有两条硬约束，都踩过 (第 8.3 节)：① 必须在 __bss_end 之后 (否则被清零 → 守卫永远不触发)；② 必须在 __heap_start 之前 (否则被 rt_system_heap_init() 当空闲内存写花)。
- KEEP 同样是必需的：s_clk_guard 只在 drv_clk.c 内部被引用，极端情况下 --gc-sections + 优化可能让它“消失”，KEEP 把这段钉住。

4.10 /DISCARD/：把不需要的段扔掉

```
/DISCARD/ : { *(.comment) *(.note*) *(.eh_frame*) *(.riscv.attributes) }
```

- 这些段在 ELF 里是**非分配 (没有 VMA)** 的元数据，objdump -h 里 VMA 显示为 0 (.debug_* 同理)。它们**不会**被 ROM 搬进 RAM，但如果不 DISCARD，某些工具链会把 .riscv.attributes 之类排进镜像造成浪费/地址错乱。
- .debug_* 没有被 DISCARD (保留调试信息，方便 gdb/addr2line 定位到源码行)。代价是 app.elf 有 2~3 MB 调试信息 (app.bin 不受影响)。

4.11 链接脚本给出的所有“魔法符号” (一张表全对上)

符号	值	定义处 (linker.ld)	谁在用
<code>__stack_top</code>	<code>0x2F07AFC0</code>	137 <code>ORIGIN+LENGTH</code>	<code>startup.S</code> 设 <code>sp</code>
<code>__stack_bottom</code> / <code>__rt_rvstack</code>	<code>0x2F078FC0</code>	138-139	<code>startup.S</code> 设 <code>mscratch</code> (<code>SW_handler</code> 中断栈顶)
<code>__irq_stack_bottom</code>	<code>0x2F077FC0</code>	140	堆上界
<code>__heap_start</code>	<code>0x2F00EF60</code>	141 = <code>__noinit_end</code>	<code>board.c</code> 的 <code>rt_system_heap_init()</code>
<code>__heap_end</code>	<code>0x2F077FC0</code>	142	同上
<code>__bss_start</code>	<code>0x2F00D588</code>	89	<code>startup.S</code> 清零循环
<code>__bss_end</code>	<code>0x2F00EF50</code>	119 <code>PROVIDE(=</code> <code>__sbss_end)</code>	同上 (注意是 PROVIDE)
<code>__sbss_start</code> / <code>__sbss_end</code>	<code>0x2F00EEC0</code> / <code>0x2F00EF50</code>	96 / 100	定义清零范围用
<code>__noinit_start</code> / <code>__noinit_end</code>	<code>0x2F00EF50</code> / <code>0x2F00EF60</code>	107 / 111	堆起点
<code>__sdata_start</code> / <code>__sdata_end</code>	<code>0x2F00D578</code> / <code>0x2F00D584</code>	83 / 85	gp 公式
<code>__rti_fn_start</code> / <code>__rti_fn_end</code>	<code>0x2F00D368</code> / <code>0x2F00D390</code>	66 / 68	(本 port 未直接用; 供遍历/调试)
<code>__fsymtab_start</code> / <code>__fsymtab_end</code>	<code>0x2F00D390</code> / <code>0x2F00D4B0</code>	73 / 75	<code>finsh_system_init()</code> 建 <code>msh</code> 命令表
<code>__global_pointer\$</code>	<code>0x2F00DD78</code>	129-130	<code>startup.S</code> 装 <code>gp</code>
<code>end</code> / <code>_end</code>	<code>0x2F00EF50</code>	133-134 <code>PROVIDE</code>	<code>newlib</code> 的 <code>sbrk</code> 约定 (本工程不实现 <code>sbrk</code> , 符号给出即可)

三条 ASSERT (链接期的“体检”) :

```
ASSERT(__noinit_end <= __heap_end,      "BSS/NOINIT OVERLAPS HEAP: image too big ...")
ASSERT(__heap_end  >= __fsymtab_end,    "HEAP OVERLAPS INIT/FINSH TABLES")
ASSERT((s31_clic_entry & 0x3F) == 0,    "s31_clic_entry MUST be 64-byte aligned for CLIC mtvec")
```

这三条不是装饰: 它们分别对应“镜像太大压到栈/堆”、“堆和表重叠”(第 8.1 节的故事)、“CLIC 入口没对齐”。改链接脚本后如果它们没报错, 说明最低限度的自治还在。想看它们怎么工作: 故意把 `LENGTH(RAM)` 改大一点再链接, `ld` 会直接把这句话打出来并失败。

5. 第四站: 符号表怎么读 (nm 的字母、addr2line 反查)

5.1 nm -n 的字母是什么意思

```
$nm -n -S build/app.elf | Select-Object -First 30
```

字母	含义	本工程里的例子
T / t	代码 (.text) , T=全局 / t=局部	T _start、t s31_clk_to_xtal_40m
R / r	只读数据 (.rodata)	R __rti_fn_start
D / d	有初值数据 (.data / .sdata)	d _rt_thread_defunct
B / b	无初值数据 (NOBITS)	b idle_thread_stack、b rt_tick
A	绝对符号 (不是内存里的东西, 只是一个数)	A WDT_WKEY (startup.S 的 .set)

⚠ 两个必须知道的坑:

1. **B 不区分 .bss / .sbss / .noinit** —— 这三个段都是 NOBITS, nm 一律显示 b/B。想分清它们在哪儿, 只能看 objdump -h 或 build\app.map。
2. 链接脚本 **PROVIDE** 出来的符号, “大小”列是垃圾值。例如:

```
2f00ef60 0006c060 B __noinit_end      ← 0x6C060 不是它的大小
2f07afc0 fff93fa0 B __stack_top      ← 负数更明显
```

因为它们不是“对象”, 只是“地址的别名”。比较可靠的做法是: **只用地址, 不用大小**。

5.2 从“运行时地址”反查源码 (排错第一技能)

发生异常时 trap_handler.c 会打印 mcause / mepc / mtval。拿到 mepc 后:

```
$addr2line -f -i -e build\app.elf 0x2f000140 0x2f00035e 0x2f003426 0x2f008e3c
```

实测输出 (真的能查到文件:行号):

```
entry
projects/rtt_nano_s31/rt-thread/src/components.c:161
main_thread_entry
projects/rtt_nano_s31/rt-thread/src/components.c:182
finsh_system_function_init
projects/rtt_nano_s31/rt-thread/components/finsh/shell.c:910
finsh_system_init
projects/rtt_nano_s31/rt-thread/components/finsh/shell.c:967
```

- 一个地址可能**内联**出多行 (-i 显示内联链) —— 这就是为什么 0x2F003426 既是 finsh_system_function_init 又是 finsh_system_init (前者被内联进后者)。
- 对**纯链接脚本符号** (如 0x2f00d588) 它只能给出名字、给不出文件: __bss_start ???:? —— 这本身就是一个有用的信号: “这个地址上没有代码/数据对象, 它是段边界”。

5.3 三种“看同一件事”的工具, 各有各的用处

工具	看什么最方便	命令
<code>objdump -h</code>	段表（地址/大小/对齐/是否 NOLOAD）	<code>\$od -h build\app.elf</code>
<code>objdump -p</code>	程序头（真正被装载的 PT_LOAD / filesz / memsz）	<code>\$od -p build\app.elf</code>
<code>objdump -d</code>	反汇编（含符号注释）	<code>\$od -d --start-address=0x2f000000 --stop-address=0x2f0000c0 build\app.elf</code>
<code>objdump -s -j 段名</code>	段的原始字节（看表、看初值、看字符串）	<code>\$od -s -j .rti_fn build\app.elf</code>
<code>nm -n -S</code>	符号（地址排序 + 大小）	<code>\$nm -n -S build\app.elf</code>
<code>size</code>	一句话总量（text/data/bss）	<code>\$size build\app.elf</code>
<code>build\app.map</code>	谁贡献了哪个段、链接器算出的每个符号值	搜 Memory Configuration / .rti_fn
<code>addr2line</code>	地址 → 源码行	见 5.2

排查顺序建议：`objdump -h` 定位段 → `map` 找责任人 → `nm` 看符号 → `objdump -d/-s` 看内容 → `addr2line` 回到源码行。

6. 第五站：gp 与小数据 —— 唯一一处“编译器 × 链接器 × 启动文件”的契约

前面第 3.3 节说“gp 没设好就去随机地址读写”，第 3.5 节又看到一条指令被链接器从两条压成一条。这一节把它量化：gp 到底值多少、省了多少、为什么现在只省这么点。

6.1 现状（本工程当前配置）

项	值
<code>__global_pointer\$</code>	<code>0x2F00DD78</code> (= <code>__sdata_start + 0x800</code> ，由 <code>MIN(...)</code> 选中)
<code>.sdata</code>	<code>0x2F00D578 .. 0x2F00D584</code> （距 gp -2048 .. -2036，刚好在 ±2 KB 边界内）
<code>.sbss</code>	<code>0x2F00EEC0 .. 0x2F00EF50</code> （距 gp +4424 .. +4568，超出 ±2 KB）
全镜像里 gp 相对寻址的指令	7 条
能靠 gp 省下的指令	<code>finsh_prompt.0</code> (+292) 3 条、 <code>_hw_pin+0x4c</code> (+500) 4 条，外加 <code>startup.S</code> 清 BSS 起点 1 条（在 <code>.boot</code> 里）

也就是说：当前这套 `MIN(__sdata_start+0x800, MAX(__sdata_end, __sbss_end)-0x800)` 让 gp 偏向了 `.sdata` 一侧，而 `.sbss` 里那 30 多个内核热点变量（`rt_tick`、`rt_interrupt_nest`、调度器锁.....）全都在 ±2 KB 之外，只能继续用 `lui+addi` 两条指令寻址。

6.2 一个实验：把 gp 的价值量出来

实验在**工程副本**里做（%TEMP%\rtt_exp_*），**没有改动仓库里的文件**。每次都是 -BuildOnly -Rebuild 全量重编。

变体	怎么改	gp	gp 相对指令	.text	app.bin
控制组（= 现状，副本重编）	不改	0x2F00DD78	7	0x96E0 / 54348	54656
A: .sbss 紧跟 .sdata	把 .sbss 段挪到 .bss 之前，并把 __bss_start 定义挪到 .sbss 开头、__bss_end 挪到 .bss 末尾	0x2F00CC98	143	0x9548 / 53940 (-408)	54256 (-400)
B: 不改顺序，只把 gp 锚定到 .sbss 末端	公式改成 MAX(...)- 0x800	0x2F00E5D0	144	0x9568 / 53976 (-372)	54288 (-368)

A 变体多出来的 gp 相对寻址覆盖了这些符号（实测）：

```
rt_tick rt_interrupt_nest rt_current_priority rt_thread_ready_priority_group
rt_scheduler_lock_nest rt_thread_switch_interrupt_flag rt_interrupt_from/to_thread
system_heap _console_device _soft_timer_list _timer_list shell __rt_errno
finsh_prompt_custom s_clk_guard_armed s_lcd_ready s_rx/tx_head/tail s_spi_iomux_mode
_syscall_table_begin/_end random_nr.0 spammed.0 s_lcd_clk_hz ...
```

三个结论：

- 1. “**小数据 + gp**” 是真的能省代码的：把 .sbss 挪到 gp 覆盖范围内，143 处访问各从两条指令变一条 —— .text 少 408 字节。
- 2. 当前省得少，是因为段序：linker.ld 的字面顺序把 .bss 夹在 .sdata 与 .sbss 之间（第 4.8(3) 节），两者相距 6.4 KB，一个 gp 无法同时覆盖。
- 3. B 比 A 多 1 条 gp 指令，却少省 36 字节 —— 说明除了 gp，段的位置变化还会影响 **分支/跳转的压缩松弛**（c.j/jal 能否用）。这条差异**没有逐条归因**，所以这里只把它当“存在二阶效应”的证据，不当成精确数字。

6.3 为什么不改（结论与理由）

方案	收益	代价/风险	结论
A: 调段序	app.bin -400 字节 (0.73%)	要同时改 __bss_start / __bss_end 的语义 (清零范围 = 两段之和), 一旦改错就是“漏清 BSS / 清掉哨兵”这类极难查的问题	✗ 本工程不做, 记录下来
B: 只改 gp 公式	-368 字节	改一行, 风险低; 但会让 .sdata 落到 ±2 KB 之外 (_rt_thread_defunct 等 反而变差)	⚠ 想做可以做, 先跑通再提交
保持现状	0	7 条 gp 指令, 正确性没有任何问题 (够不到就用绝对寻址, ABI 允许)	✅ 本工程的选择

✦ 这类“能用但没吃到”的优化, 正确的处理方式是**先量、再判断、然后写下来**, 而不是顺手改掉。这里的数字就是量的结果; doc\ 里留着它, 将来 RAM 紧张时直接拿来用。

7. 第六站: 这套约定由谁保护

光靠“小心”是不够的。本工程给“段布局约定”上了 6 道锁:

#	保护手段	在哪	保护什么
1	ASSERT(...) ×3	linker.ld 144-146	堆/栈/表/入口对齐的最低自治
2	KEEP(...)	.boot / .cllic_entry / .text.entry / .rti_fn / .fsymtab / .noinit	防 --gc-sections 删掉“没人引用但必需”的东西
3	SORT(.rti_fn*)	.rti_fn	保证级别顺序
4	链接后自检	build.ps1 打印 CLIC entry @ 0x2F0000C0 64B aligned: True	防 CLIC 入口没对齐
5	开机打印	board.c: banner + 复位原因 + 堆范围	一眼看出“是不是复位循环/堆起在哪”
6	调试开关	board.c 的 S31_DEBUG_INIT_TABLE=1 → s31_dump_init_table()	打印 .rti_fn 的 区间、边界哨兵、每一条

第 6 条的实测输出格式 (board.c 第 40-52 行) :

```
[ini] table 0x2f00d368 .. 0x2f00d390 (10 entries)
[ini] sentinels: start=2f00d368 board_start=2f00d36c board_end=2f00d370 end=2f00d38c
[ini] @2f00d368 -> 2f00035a rti_start
[ini] @2f00d36c -> 2f000390 rti_board_start
...
```

改 linker.ld / startup.S 的检查清单 (建议照做)

1. **链接日志**: 3 条 ASSERT 有没有报错? (报错就是硬失败, 不能忽略)
2. **对齐自检**: build.ps1 有没有打印 64B aligned: True?
3. **开机 banner**: 能不能正常打出 RT-Thread 版本 + 板级信息 (说明 `_start` → `entry` → `board_init` 这条链完好)?
4. **复位原因**: `reset : core0=0x..` 是不是 `0x00/0x0c` 之类正常值? (`0x12` = 超级看门狗没喂; `0x07` = MWDt; 反复出现就是启动代码/看门狗问题)
5. **堆范围**: `s31_info` / `free` 报的堆大小是不是等于 `__heap_end - __heap_start`?
6. **初始化表**: 把 `S31_DEBUG_INIT_TABLE` 打开重编一次, 看 10 条槽位是不是都在、指向的函数对不对; 顺便确认 `rt_components_init` 的起点/终点 (`0x2f00d370..0x2f00d38c`)。
7. **msh 命令**: `help` 里命令条数对不对? `lcd demo / reboot` 还能用吗? (`.fsymtab` 没被破坏)
8. **符号抽查**: `nm -n | Select-String '_start|entry|__bss_end|__heap_start|__global_pointer'` —— 和本文的表对一遍。

8. 三次真实事故复盘 (都用 ELF/map 定位)

8.1 堆把“初始化表 / 命令表”写花了 → 跳到 0x8C

- **现象**: 启动到“遍历初始化表”时 `instruction access fault`, `mepc` 附近是 `0x8C` (一个明显不是代码的地址)。
- **怎么定位**: `nm -n` 看 `__heap_start` / `__heap_end`, 发现堆的起点落在 `.rti_fn` / `.fsymtab` 之前 —— 即两张表在“堆的地盘”里。`rt_system_heap_init()` 把那一整块当空闲内存初始化, 用链表节点覆盖了表内容 → `lw a5,0(s0)` 取到的“函数指针”是垃圾值 (`0x8C`)。
- **根因**: 链接脚本里表的顺序/堆起点定义不当 (当时表放在 `.sbss` 之后, 而 `__bss_end` 只算到 `.sbss` 末尾)。
- **修法**: 凡是有符号指向的区间, 都必须落在 `__heap_start` 之前; 现在 `.rti_fn` / `.fsymtab` 明确排在 `.bss` 之前, 并加了 `ASSERT(__heap_end >= __fsymtab_end, ...)`。
- **教训**: 这类“内存被别人踩了”的问题, `objdump -h + nm` 看区间关系比读代码快得多。

8.2 --gc-sections 把 .rti_fn 整段删了 → 系统“看起来正常”但不初始化

- **现象**: 版本号、main、调度器都跑, 但所有 `INIT_*_EXPORT` 的函数一个都不执行 (tick 不走、msh 不出现、驱动全没注册)。没有任何报错。
- **怎么定位**: `objdump -s -j .rti_fn build\app.elf` → 段根本不存在 (或者大小 0); `nm` 里也找不到 `__rt_init*` 符号。
- **根因**: 这张表只被“按地址遍历” (第 4.5 节的反汇编), 链接器看不到任何引用 → 被当成无用段回收。
- **修法**: `KEEP(*(SORT(.rti_fn*)))`; `.fsymtab` 同理 (`KEEP(*(FSymTab*))`)。
- **教训**: “链接器看不见的引用”就要 `KEEP`: 自动初始化表、命令表、向量表、跳板、跨复位区。

8.3 提频守卫的哨兵被清零 → 守卫永不触发

- **现象**: `.noinit` 里的哨兵写进去, 复位后读回 0, 跳过逻辑永不生效。

- **怎么定位：** `nm -n` 把三个符号排出来 → 发现 `.noinit (0x2F00EF50)` 落在 `__bss_start..__bss_end` 区间内（当时 `__bss_end` 被定义到 `.noinit` 之后）。再看 `startup.S` 的清零循环反汇编，终点符号正是 `.noinit` 的第一个变量（第 3.5 节那张表）。
- **修法：** `PROVIDE(__bss_end = __sbss_end)`（终点停在 `.noinit` 之前），堆的起点改用 `__noinit_end`。两行改动，但语义必须精确到字节。
- **教训：**“清零范围”是一个用符号表达的内存边界。边界错 4 字节，症状会出现在完全不相干的地方（时钟守卫）。

8.4 附带一条“看反汇编别慌”的经验

反汇编里显示的符号名不一定是源码里写的那个（第 3.5 节）：`la t0, __bss_start` 显示成 `# 2f00d588 <idle_thread_stack>`，`la t1, __bss_end` 显示成 `# 2f00ef50 <s_clk_guard_skips_inv>`。同址多符号时 `objdump` 只挑一个。核对时以 `nm -n`（列全部符号）或 `build/app.map` 为准。

附录 A：一页速查（地址 ↔ 符号 ↔ 谁给的）

0x2F000000	<code>_start / .boot</code>	<code>startup.S (ENTRY(_start))</code>
0x2F0000C0	<code>s31_clic_entry / .clic_entry (64B 对齐)</code>	<code>trap_gcc.S</code>
0x2F000140	<code>entry / .text 起点</code>	<code>components.c:161</code>
0x2F000190	<code>SW_handler</code>	<code>libcpu 汇编</code>
0x2F00035E	<code>main_thread_entry</code>	<code>components.c:182 (内含 rt_components_init)</code>
0x2F005B8C	<code>rt_hw_board_init</code>	<code>bsp/board.c:118</code>
0x2F008E3C	<code>main</code>	<code>app/main.c:482</code>
0x2F009840	<code>.rodata 起点 ("main" 等字符串)</code>	
0x2F00D368	<code>.rti_fn 起点 (10 个函数指针)</code>	<code>KEEP(SORT(.rti_fn*))</code>
0x2F00D390	<code>.fsymtab 起点 (24 条 msh 命令)</code>	<code>KEEP(FSymTab)</code>
0x2F00D4B0	<code>.data (object.o 的对象容器)</code>	
0x2F00D578	<code>__sdata_start / .sdata (12 B)</code>	
0x2F00D588	<code>__bss_start / .bss (6456 B)</code>	<code>startup.S 清零起点</code>
0x2F00DD78	<code>__global_pointer\$</code>	<code>linker.ld 129</code>
0x2F00EEC0	<code>__sbss_start / .sbss (144 B)</code>	
0x2F00EF50	<code>__bss_end = __sbss_end = __noinit_start</code>	<code>startup.S 清零终点</code>
0x2F00EF60	<code>__noinit_end = __heap_start</code>	堆起点
0x2F077FC0	<code>__irq_stack_bottom / __heap_end</code>	堆终点（中断栈底）
0x2F078FC0	<code>__rt_rvstack = __stack_bottom</code>	中断栈顶（ <code>mscratch</code> 初值）
0x2F07AFC0	<code>__stack_top</code>	启动栈顶（ <code>sp</code> 初值）

附录 B：复现本文所有数据的命令

```

cd E:\esp-idf-s31\projects\rtt_nano_s31
$bin = "$env:USERPROFILE\.espressif\tools\riscv32-esp-elf\esp-15.2.0_20251204\riscv32-esp-elf\bin"
$od = "$bin\riscv32-esp-elf-objdump.exe"; $nm = "$bin\riscv32-esp-elf-nm.exe"
$sz = "$bin\riscv32-esp-elf-size.exe"; $a2l = "$bin\riscv32-esp-elf-addr2line.exe"

pwsh -File tools\build.ps1 -BuildOnly -Rebuild # 全量构建 (约 1 分钟)

& $sz build\app.elf # text/data/bss 总量
& $od -h build\app.elf # 段表 (地址/大小/对齐)
& $od -p build\app.elf # 程序头 (PT_LOAD / filesz / memsz)
& $nm -n -S build\app.elf # 符号 (按地址排序, 带大小)
& $od -d --start-address=0x2f000000 --stop-address=0x2f0000c0 build\app.elf # _start
& $od -d --disassemble=main_thread_entry build\app.elf # 初始化表消费循环
& $od -s -j .rti_fn build\app.elf # 初始化表原始字节
& $od -s -j .fsymtab build\app.elf # msh 命令表
& $od -s -j .sdata build\app.elf # 小数据段内容
& $a2l -f -i -e build\app.elf 0x2f000140 # 地址 → 源码行

# 镜像 (头/段/尾部)
python -m esptool --chip esp32s31 elf2image -fm dio -ff 80m -fs 16MB -o build\app.bin build\app.elf
# 然后按第 2 节的表解析 app.bin 的头 24 字节 + 段表

```

build\app.map 用文本搜索即可 (关键行示例) :

Memory Configuration			
RAM	0x2f000000	0x0007afc0	xrw
.rti_fn	0x2f00d368	0x28	
.rti_fn.3	0x2f00d380	0x4	...\drv_spi.o
.fsymtab	0x2f00d390	0x120	
.bss.s_tx_ring	0x2f00df70	0x800	...\drv_usj.o
.noinit	0x2f00ef50	0x10	

附录 C：术语表

术语	一句话解释	本工程实例
VMA / LMA	运行地址 / 加载地址。两者不同时，启动代码要自己搬（.data）	本工程 两者相同 （ROM 直接搬进 RAM）
NOLOAD / NOBITS	段在 ELF 里占地址但 不占文件 （没有初值）	.bss / .sbss / .noinit
PT_LOAD	ELF 程序头里“需要被装载”的段	只有 1 个：vaddr 0x2F000000, filesz 0xD584, memsz 0xEF60
gc-sections	链接器删掉“没人引用”的段/函数	表必须 KEEP，否则整段消失（8.2 节）
松弛 relaxation	链接器把伪指令/长指令换成更短的（省字节）	la → addi gp, ... ; call → jal
gp / 小数据区	用 gp 相对寻址一条指令访问 ±2 KB 内的静态量	.sdata / .sbss（第 6 节）
CLIC / mtvec	中断控制器 / 陷阱向量寄存器；模式 3 = CLIC，入口 64 B 对齐	s31_clic_entry（.clic_entry 段）
mscratch	一个 CSR，用来“进陷阱时换成中断栈”	SW_handler 第一件事就是换 sp
自动初始化表	RT-Thread 用“段收集 + 区间遍历”实现的分级初始化	.rti_fn（INIT_DEVICE_EXPORT 等）
FSymTab	msh 命令表（12 字节/条的结构体数组）	.fsymtab（24 条命令）
PROVIDE	链接脚本里“没人定义我才定义”的符号	__bss_end、end、_end

附录 D：本次核对的环境与产物

项	值
工程 / 提交	projects\rtt_nano_s31 @ 52a9e8d（工作区无未提交改动）
工具链	riscv32-esp-elf esp-15.2.0_20251204（GCC 15.2.0）
RT-Thread	Nano v5.2.2（rt-thread.commit 记录来源提交）
size build\app.elf	text 54348 / data 212 / bss 6616 / dec 61176
build\app.bin	54656 字节（4 段 + 4 字节填充 + 32 字节摘要）
ELF SHA-256	788467ddfb29dabd9214decc8a05abbefdbcf39750724da518ff32a8fcd31831（⚠ 含 __DATE__/__TIME__，重编后会变； 段表/符号/大小不会变 ）
实验副本	%TEMP%\rtt_exp_ctrl（控制组）/ rtt_exp_a / rtt_exp_b， 未进库

本文的定位：它是 doc\rt-thread-porting-tutorial.md（从空目录到 msh）的**姊妹篇**。那篇讲“怎么把 RT-Thread 跑起来”，这篇讲“跑起来的那份镜像，每一个字节为什么在那个地址”。两篇一起看，基本可以把“裸机 + RTOS + 链接脚本”这条链彻底打通。